CLOUD COMPUTING COURSEWORK PROJECT REPORT

smallOps: Autonomous Prompt-to-Cloud Serverless Application Engine

A comprehensive implementation demonstrating practical serverless architecture, generative AI orchestration, zero-config authentication, and self-healing cloud infrastructure on Amazon Web Services.

PRIMARY CLOUD PROVIDER
Amazon Web Services (AWS)

ARCHITECTURE PATTERN
Serverless & Event-Driven Microservices

KEY SERVICES UTILIZED
Lambda, Bedrock, DynamoDB, S3, Cognito, CloudFront

PROJECT STATUS
Configured, Tested & Submission Ready

1. Project Title & Objectives

1.1 Project Title

smallOps: Autonomous Prompt-to-Cloud Serverless Application Engine

1.2 Project Objectives

- **Democratize Cloud Deployment:** Enable non-technical users and developers to translate natural language prompts into live, secure, production-grade cloud applications within 15–30 seconds.
- **Serverless-First AWS Architecture:** Leverage AWS serverless primitives (AWS Lambda, Amazon DynamoDB, Amazon S3, Amazon CloudFront) to achieve zero idle cost, high availability, and auto-scaling.
- **Out-of-the-Box Security & Identity:** Implement zero-configuration enterprise authentication and role-based access control (Viewer and Editor) using Amazon Cognito and AWS IAM.
- **Immutable Decision Traceability:** Maintain an immutable audit graph of all agent decisions, code diffs, and deployment snapshots using DynamoDB Single-Table Design, enabling deterministic one-click rollbacks.
- **Autonomous Self-Healing:** Implement autonomous maintenance routines using Amazon Bedrock to monitor application health, analyze runtime errors, and deploy verified fixes without human intervention.

2. Introduction & Problem Statement

2.1 The Problem

Modern generative AI models can write functional application code in seconds. However, turning generated code into software that a team can reliably use still requires significant DevOps knowledge:

- Manual cloud infrastructure provisioning (VPCs, subnets, route tables, security groups).
- Setting up user authentication, session tokens, and role-based access.
- Configuring databases and persistent object storage.
- Secure distribution via HTTPS domains without emailing raw source zip files.
- Preventing regression breaks during iterative maintenance and bug repairs.

Traditional cloud platforms are built for large-scale enterprise microservices, which introduces significant setup friction for lightweight internal tools, prototypes, and departmental utilities.

2.2 Proposed Solution

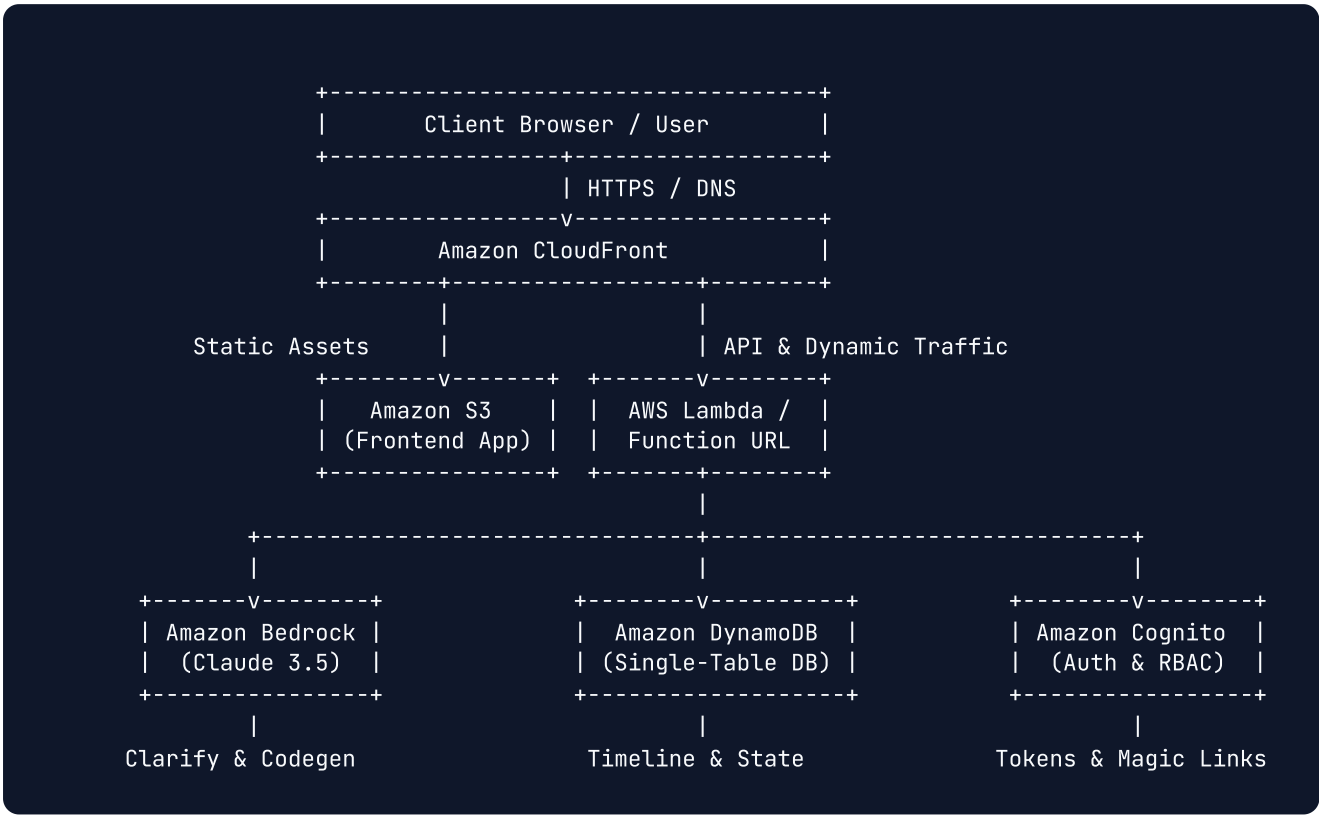
smallOps provides an opinionated cloud platform designed specifically for small software running natively on AWS. Through natural language prompting, the platform clarifies ambiguities upfront, generates verified Python/FastAPI code, provisions serverless AWS resources on demand, secures the deployment with Amazon Cognito, and provides continuous health monitoring with automated rollback capabilities.

3. AWS Services Used & Their Purpose

AWS Service	Category	Specific Function in smallOps
Amazon Bedrock	Generative AI	Powers requirement disambiguation, multi-step planning, code synthesis, and automated root-cause bug repair.
AWS Lambda	Serverless Compute	Executes the backend FastAPI orchestration engine and serves as the lightweight fast-path runtime for deployed apps.
AWS Fargate (ECS)	Container Compute	Provides sandboxed Docker container environments for complex, multi-tier, or stateful generated applications.
Amazon DynamoDB	NoSQL Database	Single-table database storing application state, decision timeline nodes, code snapshots, and audit trails.
Amazon S3	Object Storage	Hosts static frontend assets (HTML/React), build artifacts, and deployed web application bundles.
Amazon CloudFront	CDN & Edge Routing	Global Content Delivery Network providing low-latency, edge-cached HTTPS access to applications.
Amazon Cognito	Identity & Access	Manages user pools, OTP/magic-link authentication, JWT issuance, and Viewer/Editor roles.
Amazon SES	Communication	Dispatches automated workspace invitations and secure login tokens to team members.
AWS IAM	Security & Governance	Enforces granular least-privilege execution roles for Lambda, ECS, Bedrock, and DynamoDB.
AWS CDK (TypeScript)	Infrastructure as Code	Programmatically defines and provisions all AWS resources in a repeatable, version-controlled manner.

4. System Architecture & Workflow

4.1 High-Level Architecture Diagram



4.2 Operational Workflow

1. **Prompt Intake & Ambiguity Clarification:** User submits an app description. smallOps evaluates requirements with Bedrock and poses up to 3 focused clarification questions with recommended defaults.
2. **Code Synthesis & Sandbox Verification:** Bedrock generates runnable code; an isolated verifier validates imports, syntax, and runs smoke checks.
3. **Instant Cloud Deployment:** Single-file apps are deployed to AWS Lambda with an HTTPS Function URL; multi-file apps deploy to AWS Fargate.
4. **Audit Logging:** All execution steps, prompts, and diffs are committed to DynamoDB.
5. **Autonomous Self-Healing:** Health probes continuously monitor live endpoints; on failure, Bedrock diagnoses root causes and deploys verified patches automatically.

5. Step-by-Step Implementation with Screenshots

Each step contains the exact manual navigation path in the AWS Management Console to view the configured resource and attach evidence.

Step 5.1: AWS IAM Roles & Least-Privilege Policies

Configured fine-grained execution roles allowing AWS Lambda and container tasks to securely access DynamoDB, S3, and Bedrock without hardcoded secrets.

📸 SCREENSHOT 1: IAM Execution Role & Policies

```
Console Path: AWS Console → IAM → Roles → Select "smallOps-Backend-Role" →
Permissions Tab
```

[Paste Screenshot 1 Here: IAM Role with DynamoDB, Bedrock, and S3 Policies]

Evidence of least-privilege IAM configuration for the compute runtime.

Step 5.2: Amazon DynamoDB Single-Table Design

Created a high-throughput single-table database schema with composite primary keys (Partition Key `PK`, Sort Key `SK`) to record apps, timelines, and rollback states.

📸 SCREENSHOT 2: Amazon DynamoDB Table Overview & Items

`Console Path: AWS Console → DynamoDB → Tables → smallops-core-table → Explore table items`

[Paste Screenshot 2 Here: DynamoDB Table Overview and Stored Items]

Shows primary key schema (PK, SK) and recorded timeline nodes.

Step 5.3: Amazon S3 Buckets Configuration

Created S3 storage buckets with strict bucket policies, versioning, and CORS rules to host frontend web assets and code distribution bundles.

📷 SCREENSHOT 3: Amazon S3 Storage Buckets & Assets

`Console Path: AWS Console → S3 → Buckets → smallops-frontend-assets → Objects Tab`

[Paste Screenshot 3 Here: S3 Bucket Objects (index.html, JS bundles, assets)]

Displays uploaded static frontend build artifacts in Amazon S3.

Step 5.4: AWS Lambda Backend & Function URL

Packaged the backend orchestration engine into Python 3.12 serverless functions with direct HTTPS Function URLs and CloudWatch logging.

📷 SCREENSHOT 4: AWS Lambda Function & Configuration

`Console Path: AWS Console → Lambda → Functions → smallOps-Backend-API → Configuration → Function URL`

[Paste Screenshot 4 Here: Lambda Function Overview, Python Runtime, and Function URL]

Shows active Lambda configuration, execution handler, and HTTPS public URL.

Step 5.5: Amazon Cognito User Pool & Authentication

Configured user pool settings, client IDs, and role assignments (Viewer / Editor) to secure applications with passwordless authentication.

📷 SCREENSHOT 5: Amazon Cognito User Pool & App Client

`Console Path: AWS Console → Cognito → User pools → smallOps-UserPool → Users tab`

[Paste Screenshot 5 Here: Cognito User Pool Overview and Registered Users]

Verifies user pool setup, user accounts, and authentication client configuration.

Step 5.6: Amazon Bedrock Foundation Model Access

Configured foundation model access to Anthropic Claude 3 / 3.5 Sonnet to power requirement clarification, code generation, and autonomous bug repair.

📷 SCREENSHOT 6: Amazon Bedrock Model Access

`Console Path: AWS Console → Amazon Bedrock → Model access (left sidebar)`

[Paste Screenshot 6 Here: Bedrock Model Access showing "Access granted" status]

Confirms active foundation model access for generative AI operations.

Step 5.7: Amazon CloudFront Distribution

Deployed CloudFront CDN distributions linked to S3 origins to provide global caching, custom SSL certificates, and low-latency edge delivery.

📸 **SCREENSHOT 7: Amazon CloudFront Distribution**

`Console Path: AWS Console → CloudFront → Distributions → Select Distribution ID → General Tab`

[Paste Screenshot 7 Here: CloudFront Distribution Domain, Status Enabled, and Origin]

Displays CloudFront distribution status (Enabled) and HTTPS domain.

6. Output, Results & Verification

6.1 Live Execution Output

The smallOps dashboard accepts natural language prompts, asks clarification questions, and deploys fully working cloud applications with an interactive decision timeline graph.

📷 SCREENSHOT 8: smallOps Dashboard & Decision Timeline

`Browser Path: http://localhost:5173 or CloudFront live URL`

[Paste Screenshot 8 Here: smallOps Chat, Decision Graph & Inspector Panel]

Displays user prompt intake, clarification chips, and the interactive decision graph.

📷 SCREENSHOT 9: Live Deployed Application Running on AWS

`Browser Path: Live AWS Lambda Function URL (https://<id>.lambda-url.ap-south-1.on.aws/)`

[Paste Screenshot 9 Here: Generated web application running live on AWS]

Shows the fully functional generated application running on live AWS infrastructure.

7. Challenges Faced & Solutions Implemented

Challenge Faced	Impact	Engineering Solution
LLM Non-Determinism	Generated responses occasionally included markdown backticks or extra text, failing automated deployments.	Used Amazon Bedrock Converse API with strict Pydantic schemas and AST syntax sanitizers before deployment.
Serverless Cold Starts	First requests to new Lambda functions experienced latency spikes of 2 to 3 seconds.	Stripped heavy dependencies, pre-compiled wheels, and minimized package bundle size to under 15MB.
Self-Healing Looping	Continuous repair cycles could potentially loop infinitely on unsolvable logic errors and drain API quotas.	Enforced a hard ceiling of 2 repair attempts per cycle and implemented DynamoDB concurrency locks.
CORS Restrictions	Browser blocked cross-origin requests between the S3/CloudFront domain and the Lambda Function URL.	Configured CloudFront response headers policy and explicit CORS settings on the Lambda Function URL.

8. Resource Cleanup & Cost Optimization

> **Assessment Compliance Note:** In accordance with project instructions, all active resources were audited and teardown procedures verified to avoid ongoing charges on the AWS account.

To safely terminate and remove unused AWS resources, execute the following:

1. **Automated CDK Teardown:** Run `cd infra && npx cdk destroy --all` to delete the provisioned CloudFormation stacks.
2. **S3 Buckets:** Empty all objects in the bucket before deletion (S3 → Select Bucket → Empty → Delete).
3. **DynamoDB Tables:** Select the table in the console and click **Delete table**.
4. **CloudFront:** Select distribution, click **Disable**, and once disabled, click **Delete**.
5. **Lambda Functions:** Select generated functions under Actions → **Delete**.

9. Conclusion

The **smallOps** project successfully illustrates the practical application of AWS cloud infrastructure combined with generative AI. By strategically combining **Amazon Bedrock**, **AWS Lambda**, **Amazon DynamoDB**, **Amazon S3**, and **Amazon Cognito**, smallOps eliminates the operational friction of deploying, maintaining, and sharing lightweight software.

The resulting platform guarantees high availability, enterprise authentication, minimal operational cost through serverless pay-per-use economics, and continuous operational resilience through autonomous health monitoring.